Authentication

Computer Science 412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

HTTP Server is Stateless

Stateless: no memory of "transactions"

Client must supply context to server with each request. How?



50 First Dates (2004) — a server that forgets every request.

HTTP Statelessness — Video Reference



Watch the trailer!
https://www.youtube.com/watch?v=17KJk3ErIx0

UNIVERSITY

Learning Objectives

- Explain why HTTP is stateless and how Django uses session cookies to make it feel stateful
- Wire up `LoginView` and `LogoutView` with one line: `include("django.contrib.auth.urls")`
- Restrict views to authenticated users using `LoginRequiredMixin`; understand how `dispatch()` enforces the check
- Create users safely with `User.objects.create_user()`; verify passwords with `check_password()` — never store plaintext
- Register new users with a `CreateView` backed by Django's built-in `UserCreationForm`
- Link the built-in `User` model to your `Profile` model with `OneToOneField`; access via `request.user.profile`

Our Approach to Authentication

1. Use request.user in views & templates
Read request.user.is_authenticated and request.user.username anywhere in views or templates.

2. Log in and log out (built-in views)
django.contrib.auth.urls wires up LoginView and LogoutView with a single include().

3. Link User to your data models
Add a OneToOneField(User) on Profile; fetch the logged-in user's Profile via request.user.profile.

4. Restrict views to authenticated users
Use LoginRequiredMixin (or your custom subclass) to redirect unauthenticated requests to the login page.

5. Register a new user
Build a CreateView backed by UserCreationForm; Django hashes the password automatically.

HTTP Statelessness & Session Cookies

The Problem: HTTP Has No Memory

Every HTTP request is independent — the server remembers nothing between requests.

After login, how does the server know who you are on the next request?

Request 1: GET /profile/ → Server: **Who is this??**
Request 2: GET /feed/ → Server: **Who is this??**

The Solution: Session Cookies

At login, Django creates a **session** (stored in DB) and sends a **session cookie** to the browser.

The browser automatically sends this cookie with **every subsequent request**, so Django can identify the user.

Request 1: GET /profile/ + **Cookie: sessionid=abc**
Request 2: GET /feed/ + **Cookie: sessionid=abc**

Login Response — setting the cookie:

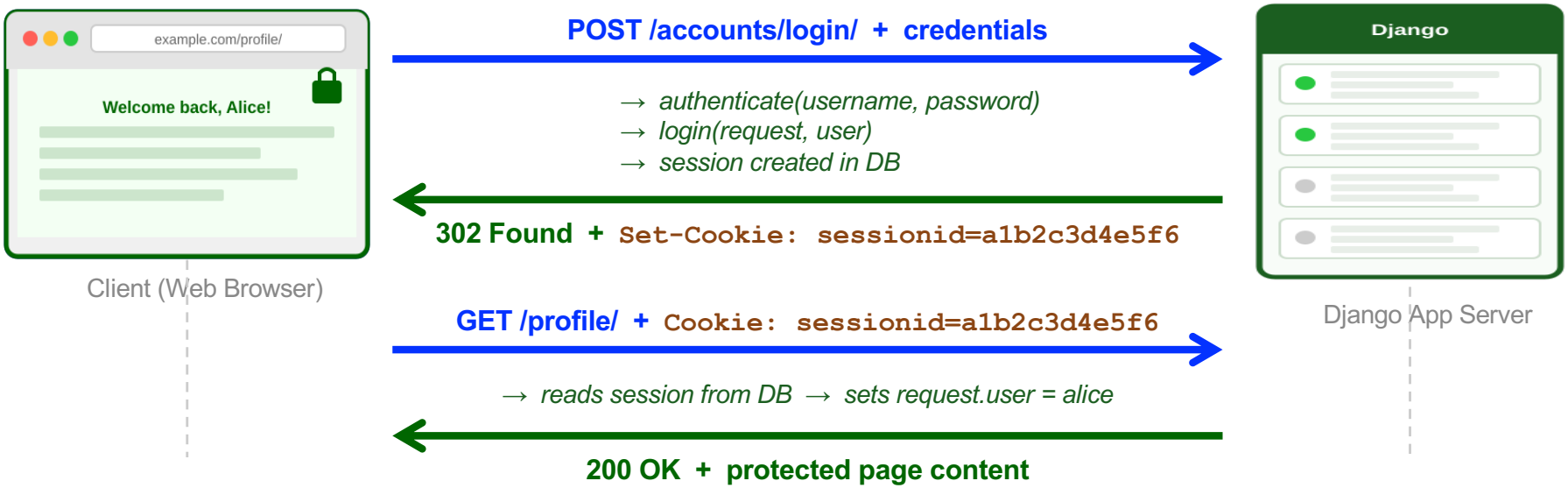
```
HTTP/1.1 302 Found
Location: /profile/
Set-Cookie: sessionid=a1b2c3d4e5f6;
            HttpOnly; Path=/; SameSite=Lax
```

Every follow-on request — browser sends the cookie:

```
GET /profile/ HTTP/1.1
Host: example.com
Cookie: sessionid=a1b2c3d4e5f6

# Django reads this → looks up session
# → sets request.user = alice
```

Django Login Flow



Login POST Request

```
POST /accounts/login/ HTTP/1.1
Content-Type: application/
  x-www-form-urlencoded

username=alice
password=••••••••
```

Login Response (Set-Cookie)

```
HTTP/1.1 302 Found
Location: /profile/
Set-Cookie:
sessionid=a1b2c3d4e5f6;
  HttpOnly; Path=/;
  SameSite=Lax
```

Follow-on Request (Cookie)

```
GET /profile/ HTTP/1.1
Cookie: sessionid=a1b2c3d4e5f6

# Django reads the cookie
# → session lookup
# → request.user = alice
```


Common Ice Cream Mix-Ins from the spruce
$ 2.99
$ 4.99
Chocolate Chips
Cookies
Candy
Fresh Fruit
Nuts
Cookie Dough

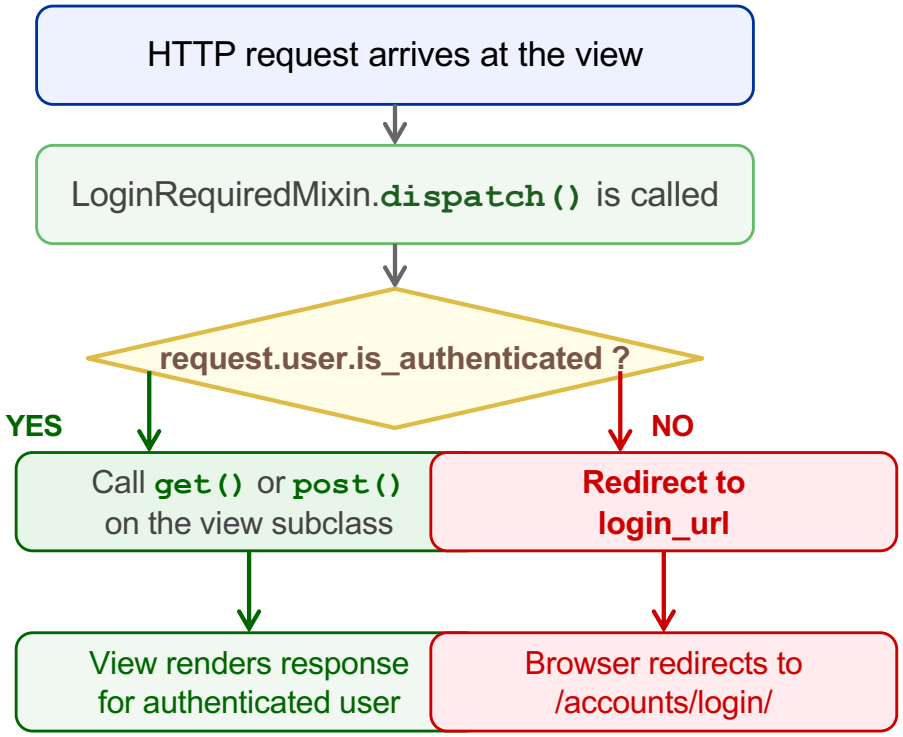
What is a “mixin”?

A mixin adds reusable behavior to a class without being the main superclass (parent). You mix it in alongside your main class — it runs first thanks to Python MRO.

```
# Pattern: mixin comes BEFORE the base view class
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView

class FeedView(LoginRequiredMixin, ListView):       # mixin first ✓
    model = Article
    template_name = 'feed.html'
```

LoginRequiredMixin — How dispatch() Works



The dispatch() method:

```
def dispatch(self, request, *args, **kwargs):
    if not request.user.is_authenticated:
        return self.handle_no_permission()
    return super().dispatch(
        request, *args, **kwargs
    )
```

Using it in your views:

```
from django.contrib.auth.mixins \
    import LoginRequiredMixin

class FeedView(LoginRequiredMixin, ListView):
    model = Article
    template_name = 'feed.html'
    login_url = '/accounts/login/'
```

The Built-in User Model

User Model Fields

`username` — login handle · unique · max 150 chars

`email` — optional · not unique by default

`password` — hashed string · never plaintext

`is_authenticated` — always True for a real User

`is_staff` — True grants /admin/ access

Creating & Retrieving

```
from django.contrib.auth.models import User

# Always use create_user() — hashes password
user = User.objects.create_user(
    username='alice',
    email='alice@example.com',
    password='s3cret',
)

# Retrieve and inspect
u = User.objects.get(username='alice')
print(u.is_authenticated)  # True
print(u.is_staff)          # False
```

Accessing in View Function

```
def my_view(request):
    u = request.user
    if u.is_authenticated:
        name = u.username   # 'alice'
        # u.email, u.is_staff …

    # AnonymousUser: is_authenticated → False
```

Linking User to Profile (OneToOneField)

A `OneToOneField` to `User` ensures each `Profile` belongs to exactly one `User` and each `User` has at most one `Profile`. Django adds `request.user.profile` as a reverse accessor.

```
# mini_insta/models.py
from django.contrib.auth.models import User
from django.db import models

class Profile(models.Model):
    user        = models.OneToOneField(User, on_delete=models.CASCADE)
    bio         = models.TextField(blank=True)
    # … other fields

    def __str__(self):
        return f'Profile of {self.user.username}'

# Access the Profile from a view:
profile = request.user.profile

# Access the User from a Profile instance:
user = profile.user
```

django.contrib.auth.urls

The One-Liner Include

```
# urls.py
from django.urls import path, include

urlpatterns = [
    path('accounts/', include('django.contrib.auth.urls')),]
```

Auth & Password URLs

accounts/login/ → LoginView

accounts/logout/ → LogoutView

accounts/password_change/ → PasswordChangeView

accounts/password_change/done/ → PasswordChangeDoneView

Password Reset URLs

accounts/password_reset/ → PasswordResetView

accounts/password_reset/done/ → PasswordResetDoneView

accounts/reset/{uidb64}/{token}/ → PasswordResetConfirmView

accounts/reset/done/ → PasswordResetCompleteView

User Registration: UserCreationForm

`UserCreationForm` is Django's built-in form for creating a `User`. Pair it with a `CreateView` — no custom `form_valid()` needed; Django hashes the password automatically.

```
# blog/views.py
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from django.views.generic.edit import CreateView
from django.urls import reverse_lazy

class RegisterView(CreateView):
    model         = User
    form_class    = UserCreationForm
    template_name = 'registration/register.html'
    success_url   = reverse_lazy('login')
```

```
# project/urls.py
from django.urls import path, include
from blog.views import RegisterView

urlpatterns = [
    path('accounts/', include('django.contrib.auth.urls')),
    path('accounts/register/', RegisterView.as_view(), name='register'),
]
```

✏️ **`UserCreationForm`** validates that both passwords match and enforces Django's built-in validators. The password is hashed automatically on `form.save()`.

Password Hashing

Hash Format

`pbkdf2_sha256$720000$salt$hash`

pbkdf2_sha256 — hashing algorithm

720000 — iteration count (cost factor)

salt — random per-user salt string

hash — Base64-encoded PBKDF2 digest

Django never stores the raw password.

Higher iterations = slower brute-force, but also slower login. Django auto-upgrades on next successful login when settings change.

Verifying Passwords

```
from django.contrib.auth.hashers import
check_password
# Django calls this internally at login:
ok = check_password('s3cret', user.password)
# ok → True if the password matches

# !! NEVER compare directly:
# user.password == 's3cret'  ← WRONG
```

Changing Passwords

```
# Re-hash and save:
user.set_password('new_pass')
user.save()

# authenticate() wraps check_password():
from django.contrib.auth import authenticate
u = authenticate(request,
    username='alice', password='s3cret')
# u is the User object or None
```

request.user in Templates

Template Conditionals

```
{# user is auto-injected via
RequestContext #}

{% if user.is_authenticated %}
  <p>Hello, {{ user.username }}!</p>
  <a href="{% url 'logout' %}">Log
out</a>
{% else %}
  <a href="{% url 'login' %}">Log in</a>
{% endif %}
```

Key template variables:

`user.is_authenticated` — True / False

`user.username` — the login name

`user.email` — the email address

`user.is_staff` — admin access

Full Template Example

```
{% extends "base.html" %}
{% block content %}

<h2>{{ user.username }}'s Feed</h2>

{% for post in posts %}
  <div class="post">
    <strong>{{ post.author.username }}
</strong>
    <p>{{ post.caption }}</p>

    {% if user == post.author %}
      <a href="{% url 'delete' post.pk %}">
        Delete
      </a>
    {% endif %}
  </div>
{% endfor %}

{% endblock %}
```

CSRF + Auth: Better Together

The Login Form

```
<form method="post">
  {% csrf_token %}
  <input name="username" type="text">
  <input name="password" type="password">
  <button>Log in</button>
</form>
```

Without this, a forged form on evil.com could POST using the victim's session cookie.

Why Both Matter

LoginRequiredMixin gates the page:
only authenticated users ever reach the form.

{% csrf_token %} guards the submit:
each POST must carry a token tied to the user's session — forged requests lack it.

Remove either one and you have a gap.

Protecting the View

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin
)

class FeedView(LoginRequiredMixin,
ListView):
    model          = Post
    template_name  = 'feed.html'
    login_url      = '/accounts/login/'
```

dispatch() runs before get() / post():

```
def dispatch(self, request, *args,
**kwargs):
    if not request.user.is_authenticated:
        return self.handle_no_permission()
    return super().dispatch(...)
```

!! Order matters: LoginRequiredMixin must come before the view class (Python MRO).

Logout Flow

Logout Template

```
{# In your template #}
<form method="post"
      action="{% url 'logout' %}">
  {% csrf_token %}
  <button>Log out</button>
</form>
```

Must be POST — GET is silently ignored
in Django 4.1+ (see warning below).

Django 4.1+ Change

!! Django 4.1+ changed logout to POST-only.

A GET to /accounts/logout/ is a no-op.

This prevents CSRF-driven forced logouts:
a malicious <img src="/accounts/logout/"> or link would previously log out any user who visited the attacker's page.

What Django Does

1. Verifies the CSRF token on the POST

2. Calls `request.session.flush()` — deletes the session row from the database

3. Creates a fresh empty session (new key)

4. Sends an expired `Set-Cookie` header:

```
Set-Cookie: sessionid=; Max-Age=0; Path=/
```

→ *browser deletes the cookie*

5. Redirects to `settings.LOGOUT_REDIRECT_URL`

After logout: session row gone, cookie expired.
The old sessionid is permanently invalidated.

Inheriting from LoginRequiredMixin

The Problem

In mini_insta, many views need both:

- Authentication check (login required)
- The logged-in user's Profile object
- The login_url set consistently

Affected views: CreateProfile, CreatePost, ShowFeedListView, …

Copying auth boilerplate into each view violates DRY.

The Solution

Subclass LoginRequiredMixin once and handle everything centrally:

Sets login_url in one place
get_logged_in_profile() fetches the Profile
get_context_data() injects profile into every template automatically

Any view that inherits MyLoginRequiredMixin gets all three for free.

Inheriting from `LoginRequiredMixin`

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import View

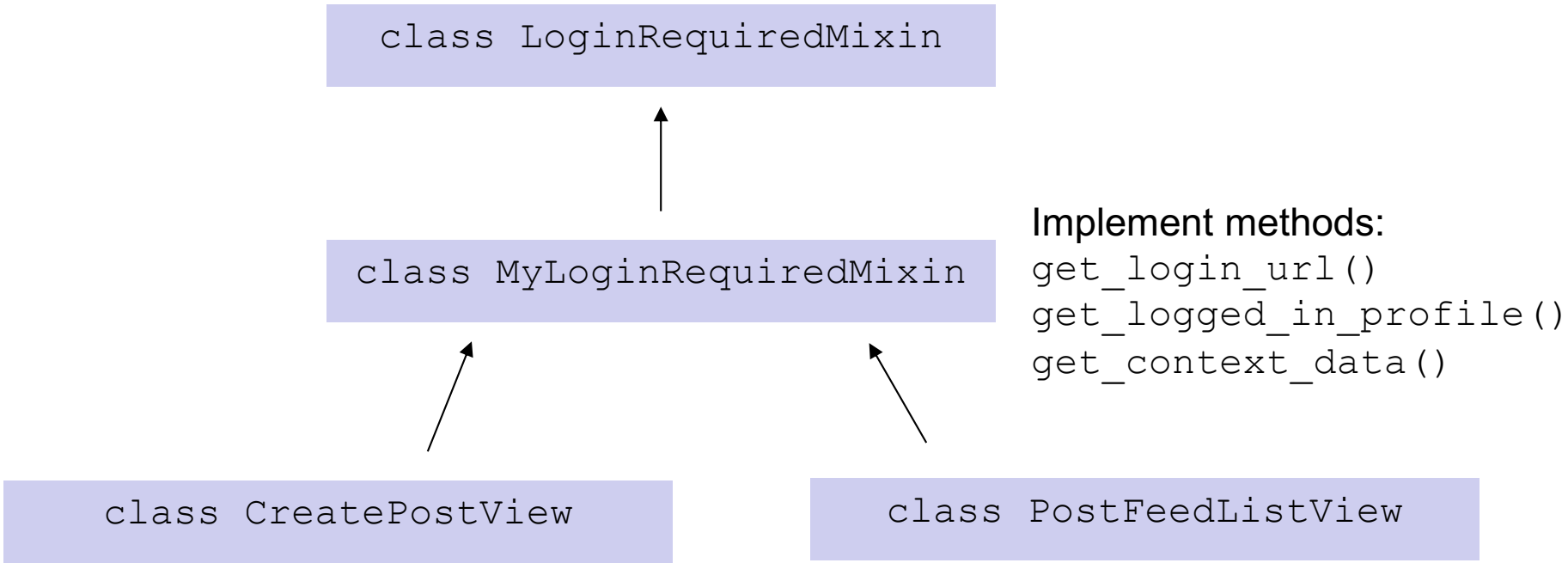
class MyLoginRequiredMixin(LoginRequiredMixin):
    login_url = '/accounts/login/'

    def get_logged_in_profile(self):
        return Profile.objects.get(user=self.request.user)

    def get_context_data(self, **kwargs):
        context = []
        context['profile'] = self.get_logged_in_profile()
        return context

class PostFeedListView(MyLoginRequiredMixin, ListView):
    model = Post
    template_name = 'feed.html'
```

Inheriting from `LoginRequiredMixin`



Additional Benefits: Cleaner URLs

Before (pk in URL)

Views needed the Profile pk to look up the user's profile:

```
profile/<int:pk>/update
profile/<int:pk>/feed
profile/<int:pk>/create_post
```

Anyone who knows another user's pk could visit their feed URL.

After (no pk needed)

MyLoginRequiredMixin provides the Profile from request.user:

```
profile/update
profile/feed
profile/create_post
```

URLs are simpler and no pk is exposed. Authentication enforced by the mixin.

Additional Benefits: Two-Profile Features

Follow another Profile

actor: request.user's Profile | target: another Profile (from URL pk)

```
Request: POST /profile/<int:pk>/add_follower
```

Like a Post

actor: request.user's Profile | target: a specific Post (from URL pk)

```
Request: POST /post/<int:pk>/like
```

Comment on a Post

actor: request.user's Profile (from mixin) | target: a specific Post (from URL pk)

```
Request: POST /post/<int:pk>/comment
```

Summary

- HTTP is stateless; Django adds statefulness via a `sessionid` cookie that maps to a DB row and sets `request.user`
- `include("django.contrib.auth.urls")` wires up `LoginView`, `LogoutView`, and password-reset views at `accounts/`
- `LoginRequiredMixin` guards views via `dispatch()` — unauthenticated requests redirect to `login_url` before any view logic runs
- `User.objects.create_user(username, email, password)` hashes automatically; use `check_password()` or `authenticate()` to verify
- Register users with `CreateView` + `UserCreationForm`; link to your app data via `OneToOneField(User, on_delete=models.CASCADE)`
- Subclass `LoginRequiredMixin` once to centralise `login_url`, profile lookup, and context injection across all protected views